



Universität von Leipzig
Fakultät für Mathematik und Informatik

Qualitätsbewertung von Datenintegrations-Benchmark- Datensätzen für Knowledge Graphs

Entwurfsdokument für das Modul
Big Data Praktikum

eingereicht von: Aische Spieker und Berkay Özcekic

Betreuer: Marvin Hofer

Leipzig, 1. Juni 2026

1 Thema

Das Dokument beschreibt den konzeptionellen Entwurf eines Python-Frameworks, mit dem wir Knowledge-Graph-Benchmark-Datensätze für Entity Alignment systematisch auf ihre Qualität untersuchen. Wir legen die zu berechnenden Basis-, Struktur- und Qualitätsmetriken fest, beschreiben Aufbau und Datenfluss der Anwendung (Pandas und PySpark) und begründen die Auswahl der sieben Datensätze, auf denen das Framework anschließend evaluiert wird.

Das gesamte Projekt ist verfügbar unter <https://github.com/aischesp/Evaluation-of-Data-Integration-Benchmark-Datasets-for-Knowledge-Graphs.git>

2 Motivation

Beim Aufbau integrierter Knowledge Graphs (KGs) spielt Entity Alignment eine zentrale Rolle: Entitäten aus verschiedenen Datenquellen, die dasselbe Realweltobjekt beschreiben, müssen erkannt und zusammengeführt werden. Um entsprechende Verfahren zu evaluieren, gibt es eine wachsende Zahl von Benchmark-Datensätzen, die jeweils zwei oder mehr KGs zusammen mit einem Referenz-Alignment bündeln. Übliche Quellen sind OpenEA [1, 2] und OAEI [3].

Die Aussagekraft eines Benchmarks hängt allerdings stark von seinen Eigenschaften ab. Ist der Graph dicht oder fragmentiert? Wie groß ist der Long-Tail an Entitäten mit nur ein bis zwei Tripeln? Sind die zwei verglichenen KGs schematisch ähnlich, oder weichen die Properties stark voneinander ab? Welcher Anteil der Alignments ist eindeutig, welcher mehrdeutig? Solche Größen bestimmen, ob ein Verfahren auf einem Benchmark überhaupt erfolgreich sein kann, und sie machen Ergebnisse über mehrere Benchmarks hinweg vergleichbar. In den Datensatzbeschreibungen werden sie trotzdem kaum dokumentiert. Mit unserem Framework wollen wir hier ansetzen.

3 Ziel und Aufgaben

Unser Ziel ist ein Python-Framework, das die genannten Eigenschaften reproduzierbar über mehrere Benchmark-Datensätze hinweg berechnet, exportiert und vergleichbar macht. Daraus ergeben sich vier zusammenhängende Aufgaben:

- Heterogene Eingabeformate (OpenEA-Tripel, RDF/OWL, CSV, eigene Subsets) in eine einheitliche interne Repräsentation überführen.
- Einen Katalog aus Basis-Statistiken, strukturellen und qualitativen Metriken konzipieren und implementieren.
- Die Berechnung skalierbar gestalten, sodass kleine Datensätze (15K Entitäten) lokal in Pandas laufen, größere Varianten (100K) in PySpark.
- Ergebnisse vergleichend darstellen, sowohl pro Datensatz als auch in einer aggregierten Übersicht.

4 Auswahl der Benchmark-Datensätze

Wir betrachten sieben Datensätze und wählen sie entlang von Größe, Sprache, Schema-Heterogenität, Dichte und Alignment-Eigenschaften aus, sodass das Framework auf einer realistischen Bandbreite getestet wird. Die ausführliche Begründung steht in `docs/Datensätze.md`. Die folgende Übersicht fasst die Auswahl zusammen.

Sechs der sieben Datensätze stammen aus OpenEA, da OpenEA als De-facto-Standard ein einheitliches Tab-getrenntes Format und vorbereitete Train/Valid/Test-Splits liefert. Innerhalb dieser Familie isolieren wir paarweise eine Variable: V1 gegen V2 zeigt den Effekt der Dichte,

#	Datensatz	# Ent.	# Tripel	Charakter
1	OpenEA EN_FR_15K_V1	30 K	~96 K	cross-lingual EN-FR, sparse
2	OpenEA EN_FR_15K_V2	30 K	~177 K	cross-lingual EN-FR, dense (Vergleich zu V1)
3	OpenEA EN_DE_15K_V1	30 K	~96 K	cross-lingual EN-DE
4	OpenEA D_W_15K_V1	30 K	~78 K	DBpedia, Wikidata; heterogenes Schema
5	OpenEA D_Y_15K_V1	30 K	~91 K	DBpedia, YAGO; reichere Typhierarchie
6	OpenEA EN_FR_100K_V1	200 K	~650 K	Skalierungsvergleich, Spark
7	OAEI Conference	~150	~600	OWL-Format, sehr klein, Sanity-Check

Tabelle 1. Übersicht der sieben Benchmark-Datensätze

EN-FR gegen EN-DE den der Sprache, D-W gegen D-Y die Schema-Heterogenität, 15K gegen 100K die Skalierung. Ergänzend nehmen wir den OAEI Conference Track (OWL-Format, sehr kleiner Graph) als Edge-Case auf.

5 Metriken

Der vollständige Katalog mit Formeln liegt in `docs/Metriken-Katalog.md`. Hier nur die drei Gruppen im Überblick.

5.1 Basis-Statistiken

Pro Knowledge Graph berechnen wir die Anzahl der Entitäten, Relations- und Attribut-Properties, Literale und Tripel sowie das Verhältnis von Attribut- zu Relations-Tripeln. Pro Paar zusätzlich die Anzahl der Alignments und das Alignment Ratio, also das Verhältnis der Alignments zur kleineren der beiden Entitätenmengen. Diese Werte sind die Grundlage jeder weiteren Auswertung und liefern den ersten Eindruck der Größenordnung eines Datensatzes.

5.2 Strukturelle Metriken

Für jede Entität ermitteln wir in-, out- und total-degree auf den Relations-Tripeln. Aus der Degree-Sequenz extrahieren wir ein log-skaliertes Histogramm und die übliche Kennzahlen-Sammlung (Mean, Median, Std, Min, Max, 25/75/95/99-Perzentile). Zusätzlich geben wir den Anteil isolierter Knoten (Degree 0) und den Anteil strukturell schwacher Knoten (Degree 2) explizit aus. Auf der Grad-Sequenz fitten wir per Maximum-Likelihood einen Power-Law-Exponenten nach Clauset et al. [5]. Der Median ist inhaltlich besonders interessant: Embedding-Verfahren brauchen genug Tripel pro Entität, um stabile Vektoren zu lernen. Fällt der Median niedrig aus, ist der Benchmark strukturell schwach.

Daneben werten wir die Property-Frequenz-Verteilung aus (welche Relationen dominieren, wie stark konzentriert sich das Volumen auf die zehn häufigsten Properties) sowie Connectivity-Maße: Anzahl der zusammenhängenden Komponenten, Anteil der größten Komponente, Dichte. Bei den großen Graphen approximieren wir Connectivity auf einer Stichprobe, damit die Laufzeiten beherrschbar bleiben.

5.3 Qualitätsmetriken

Die Qualitätsmetriken sind der inhaltliche Kern und gliedern sich in fünf Themen.

Attribut-Vollständigkeit: pro Attribut-Property der Anteil der Entitäten, die diesen Wert tragen, aggregiert über Mean und Median. Für aligned Entitäten messen wir zusätzlich, wie oft beide Partner Werte für vergleichbare Properties haben. So werden Property-Mapping-Lücken sichtbar.

Typen-Verteilung: Histogramm der `rdf:type`-Klassen, Schiefe, Jaccard zwischen den Klassenmengen der beiden KGs sowie eine Type-Consistency-Rate, die angibt, wie oft aligned Entitäten denselben Typ tragen.

Long-Tail-Analyse: kumulative Verteilung der Entity-Frequenzen, Gini-Koeffizient auf der Degree-Verteilung und Anteil der Entitäten mit $\text{Total-Degree} \leq k$. Ein hoher Long-Tail-Anteil verzerrt die Testgenauigkeit von Embedding-Methoden systematisch.

Alignment-Metriken: Coverage, Ambiguität (1:n und n:1), Anteil bijektiver Mappings sowie der Anteil aligned Entitäten innerhalb der größten verbundenen Komponente. Letzteres deckt isolierte Cluster auf, die für das Alignment-Lernen wenig Nutzen haben.

Konsistenz aligned Entitäten: Spearman-Korrelation der Degrees zwischen den beiden Partnern, Jaccard auf den Typmengen und auf Sub-Sample-Basis ein Vergleich der Attribut-Werte (numerische Distanz, Levenshtein für Strings).

Optional, je nach Zeitbudget, ergänzen wir eine sechste Gruppe Schema-Heterogenität: Property-Jaccard, Class-Jaccard, Namespace-Overlap und Entropie der Property-Verteilung.

6 Architektur

Das Framework folgt einer ETL-Pipeline mit klar getrennten Stufen. Sie liest deklarativ aus einer YAML-Konfiguration und schreibt strukturierte Outputs unter `results/reports/`.

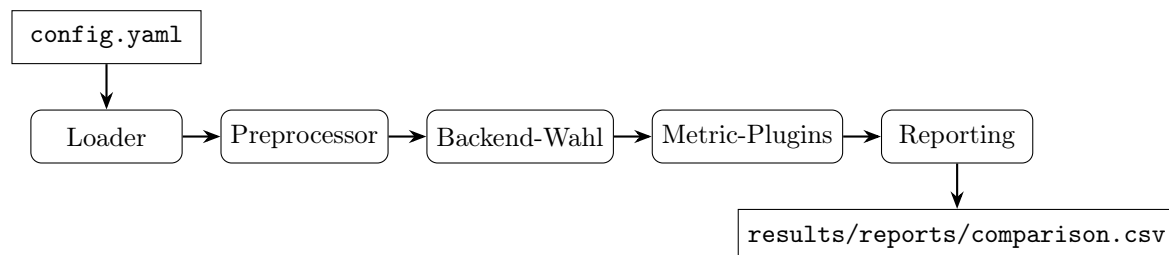


Abbildung 1. Pipeline-Übersicht

Die Loader liefern eine KGPair-Datenstruktur mit zwei KnowledgeGraph-Instanzen (Entitäten, Relations-Tripel, Attribut-Tripel) und einer Tabelle für die Alignments inklusive Train/Valid/Test-Spalten. Auf der Pandas-Seite sind die einzelnen Tabellen `pd.DataFrame`-Objekte, auf der Spark-Seite `Spark-DataFrames` mit identischem Schema. Die Metriken sind gegen diese Schnittstelle programmiert und bleiben so unabhängig vom Backend.

Auf Code-Ebene teilt sich das Framework in fünf Module. `loaders/` enthält die Basisklasse `Base-Loader` und die konkreten Implementierungen für OpenEA, RDF/OWL und CSV. `preprocessing/` übernimmt Cleaning, URI-Normalisierung und Validierung. `metrics/` enthält `BaseMetric` und je eine Datei für die drei Metrikgruppen. `reporting/` ist für JSON-/CSV-Export, Plots und die Comparison-Table zuständig. `backend/` kapselt die Pandas- und Spark-spezifische Logik. Den Pipelinekern bildet `runner.py`. Neue Datensätze, Metriken oder Backends fügen wir durch Subclassing der jeweiligen Basisklasse hinzu, ohne `runner.py` zu ändern.

Die Wahl des Backends hängt von der Datenmenge ab. Solange die Gesamtmenge der Tripel unter etwa einer Million liegt, was für sechs unserer sieben Datensätze gilt, rechnen wir mit Pandas; das ist schneller und einfacher zu debuggen. Für die 100K-OpenEA-Variante nutzen wir PySpark im

Local-Mode. Teuer sind vor allem Connectivity und Clustering. Bei großen Graphen approximieren wir sie auf einer Stichprobe von rund 10 000 Knoten, ein in der Graph-Analytik-Literatur üblicher Kompromiss.

7 Werkzeuge

Wir implementieren in Python 3.11 mit Pandas 2 und PySpark 3.5 für die Tabellen- und Cluster-Operationen, rdflib für das Parsen von RDF, Turtle und OWL und NetworkX für Connectivity-Berechnungen. Plots erzeugen wir mit Matplotlib und Seaborn, Konfigurationen liegen als YAML vor, Tests in pytest, explorative Auswertungen in Jupyter-Notebooks.

8 Zeitplan

Phase	Zeitraum	Inhalt
1	Mai 2026	Repository, Konzeptsdokumente (dieses Dokument)
2	KW 23–24	OpenEA- und RDF-Loader, Pandas-Backend, Basis-Statistiken
3	KW 25–26	Strukturelle Metriken, erste Plots
4	KW 27–28	Qualitätsmetriken (Attribut-Coverage, Alignment)
5	KW 27–28	PySpark-Backend, 100K-Datensatz
6	KW 29	Reporting, Comparison-Table, Testat 2
7	KW 30–35	Evaluation, schriftliche Auswertung
8	tbd	Testat 3 (Abschlusspräsentation)

Tabelle 2. Arbeits- und Zeitplan

9 Risiken

Der OpenEA-Download umfasst rund ein Gigabyte und wird nicht ins Repository hochgeladen. Unser Setup-Skript legt ihn unter `data/raw/openea/` ab. PySpark unter macOS kann beim Setup zu Problemen führen, dafür dokumentieren wir einen Docker-Fallback. Das OAEI-Material liegt im OWL-Format vor. Falls rdflib damit Probleme bekommt, konvertieren wir vorab nach N-Triples und werten dann mit dem RDF-Loader aus.

Literatur

- [1] Sun, Z., Hu, W., Li, C. (2017): *Cross-lingual Entity Alignment via Joint Attribute-Preserving Embedding*. arXiv:1708.05045.
- [2] Sun, Z. et al. (2020): *A Benchmarking Study of Embedding-based Entity Alignment for Knowledge Graphs*. PVLDB 13(11), 2326–2340.
- [3] OAEI (2023): *Ontology Alignment Evaluation Initiative*. <https://oei.ontologymatching.org/>
- [4] Zhang, W. et al. (2022): *An Experimental Study of State-of-the-Art Entity Alignment Approaches*. IEEE TKDE.
- [5] Clauset, A., Shalizi, C. R., Newman, M. E. J. (2009): *Power-Law Distributions in Empirical Data*. SIAM Review 51(4), 661–703.
- [6] OpenEA Repository (2020): <https://github.com/nju-websoft/OpenEA>